07

Part 7 · Databricks Platform Tuning

Cluster choices and platform features that change speed and cost.

- Cluster sizing
- Autoscaling
- Photon
- Job vs all-purpose
- Cluster pools
- Runtime & configs
- Disk cache · liquid clustering · predictive optimization

7.1 Cluster Sizing

★★★

IN ONE LINE

Choosing node types, worker count and driver size from evidence in the Spark UI.

A Databricks cluster has one **driver** and many **workers**. Sizing is choosing **node type** (CPU/RAM/disk profile), **worker count**, and **driver size**. There is no universal answer: use evidence from the Spark UI.

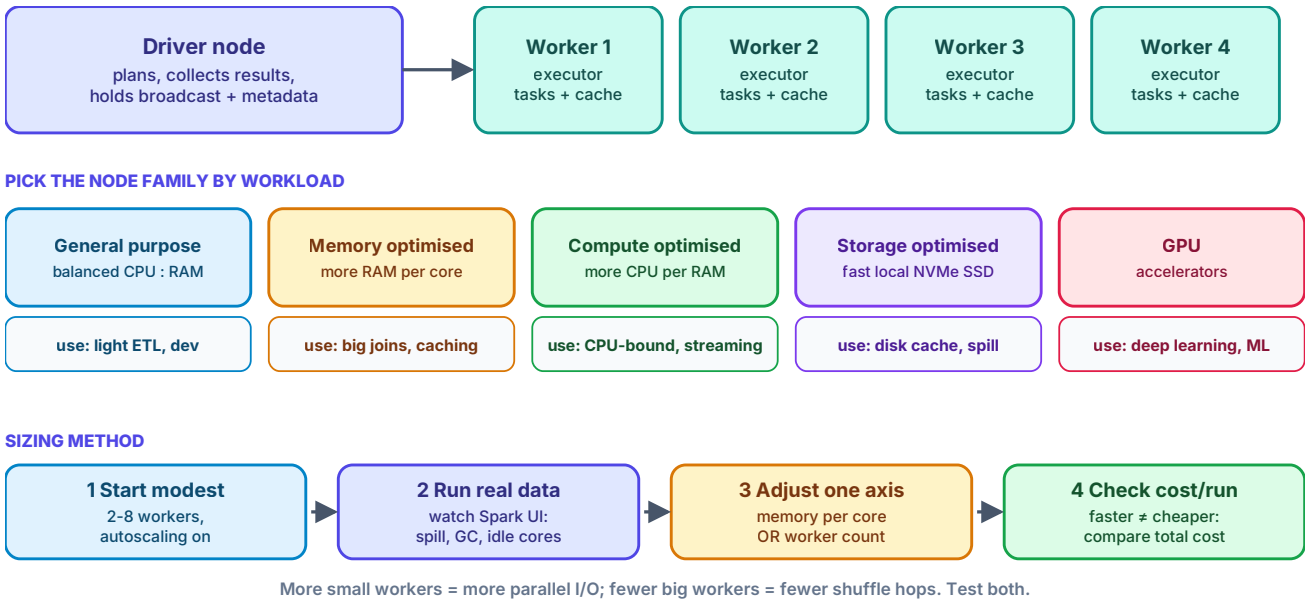


Figure 7.1 — Driver and workers, node families, and a sizing method

Symptom in Spark UI	Resize in this direction
Spill to disk, OOM, high GC	**Memory-optimised** nodes or fewer cores per node (more RAM per task)
CPUs saturated, no spill	More cores: compute-optimised nodes or more workers
Repeated reads of the same data are slow	Storage-optimised nodes (disk cache, 7.7)
Many idle cores, few tasks	Fewer workers: you cannot use them (partition count / skew)
Driver OOM / slow planning	Larger driver; reduce `collect()`, huge broadcasts, or tiny-file counts

INTERVIEW ANGLE

A mature answer: *"I do not guess: I start small with autoscaling, look at spill, GC and utilisation in the Spark UI, then change one dimension. I judge by **cost per run**, not just runtime."* More workers do not fix skew or tiny-partition problems.

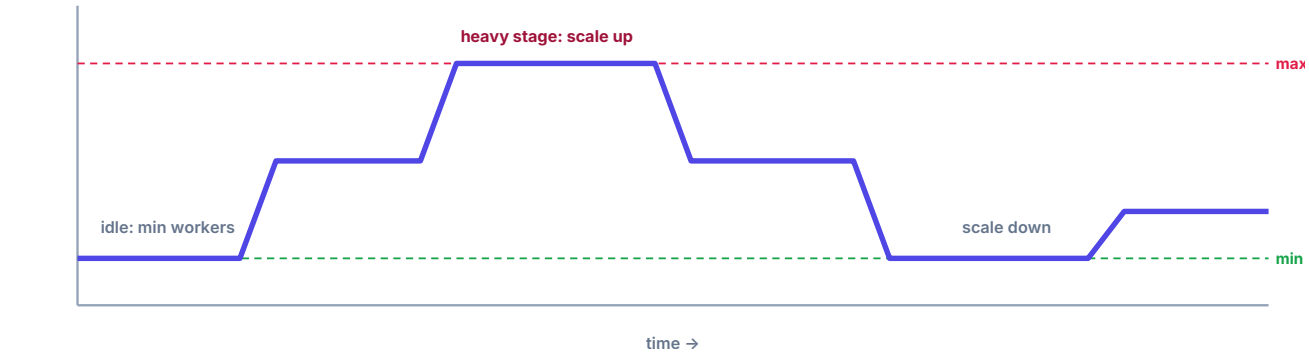
7.2 Autoscaling

★★☆

IN ONE LINE

Workers are added or removed with load between a minimum and maximum.

Autoscaling lets Databricks add workers when there are many pending tasks and remove them when they sit idle, within a **minimum and maximum** you set. You avoid paying for peak capacity all day while still handling bursts.



KEY POINTS

- **Pay for what you use** — idle workers are removed within the min/max band
- **Good for** — uneven workloads, interactive, varying data volume
- **Be careful** — shuffle-heavy jobs: removing a worker loses its shuffle files
- **Streaming** — prefer pipeline-level autoscaling (Lakeflow); fixed size otherwise

Figure 7.2 — Worker count follows demand between min and max

Setting	Advice
Min workers	Enough that a typical stage does not start tiny; very low min can make jobs ramp slowly.
Max workers	A cost guardrail and the ceiling for bursts.
Fixed-size cluster	Predictable runtime, simple to reason about; good for benchmarks and steady streaming.
Spot instances for workers	Cheap, but can be reclaimed; keep the **driver on-demand**.

COMMON TRAP

Benchmark with a **fixed** cluster. If autoscaling changes worker count between runs, you cannot tell whether your code change or the cluster size made it faster.

INTERVIEW ANGLE

Autoscaling helps with **spare capacity**, not with a bad plan: skew or tiny partitions leave added workers idle. Also remember scale-up takes time (VM start), so very short jobs may not benefit.

7.3 Photon Engine

★★☆

IN ONE LINE

Databricks' vectorised C++ engine that accelerates supported SQL and DataFrame operators.

Photon is Databricks' native, **vectorised query engine written in C++**. It replaces parts of Spark's JVM execution for supported SQL and DataFrame operators, with no change to your code or APIs. Spark still plans the query; Photon executes the supported pieces much faster.

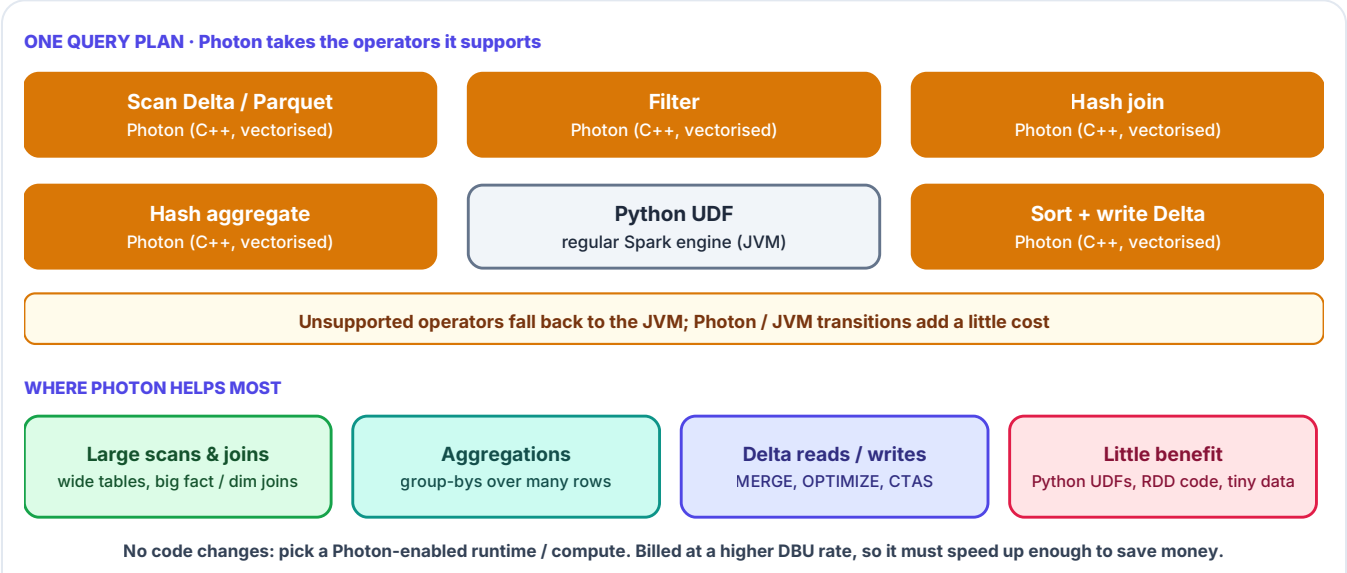


Figure 7.3 — Photon runs supported operators; the rest stay on the JVM engine

Question	Answer
How do I enable it?	Select a **Photon-enabled** Databricks Runtime when creating compute (or use SQL warehouses / serverless, where it is standard).
What speeds up?	Scans, filters, joins, aggregations, sorts, Delta writes: data-heavy SQL / DataFrame work.
What does not?	Python UDFs, RDD code, very small data, or workloads dominated by waiting on I/O.
Cost?	Higher DBU price per hour: judge by **cost per job**, compare with and without Photon.
How do I verify?	Spark UI / query profile shows **Photon** operators (e.g. PhotonGroupingAgg, PhotonShuffledHashJoin).

INTERVIEW ANGLE

Do not say "Photon makes everything faster". Say: *"Photon accelerates supported SQL/DataFrame operators; I benchmark cost and runtime on my workload and avoid UDFs so more of the plan can use it."*

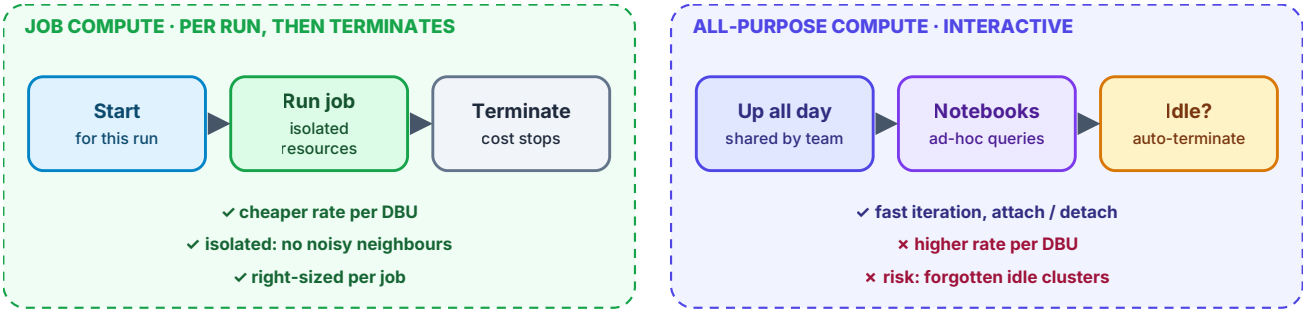
7.4 Job Clusters vs All-Purpose Compute

★★★

IN ONE LINE

Use job compute for scheduled production runs and all-purpose compute for interactive development.

Databricks offers different **compute types**. Picking the right one is mostly about **cost and isolation**: production pipelines should not run on the shared interactive cluster someone left on.



	Job compute	All-purpose compute	Serverless
Best for	Scheduled production	Development, exploration	Zero-management jobs / SQL
Lifecycle	Created per run	Long-lived, shared	Managed by Databricks
Cost	Lowest per DBU	Higher per DBU	Usage-based, no idle VMs
Startup	Cluster start each run (pool helps)	Already running	Fast

Figure 7.4 — Job compute vs all-purpose compute

Use job compute (or serverless jobs) for

- Scheduled / triggered production workflows.
- Anything that must be isolated and reproducible.
- Cost control: pay only for the run.

Use all-purpose compute for

- Developing and debugging notebooks.
- Ad-hoc exploration.
- Always set **auto-termination** (e.g. 30–60 min idle).

COMMON TRAP

Running a nightly production job on an all-purpose cluster is a classic cost mistake: higher DBU rate, resource contention with interactive users, and a cluster that stays up all day. Move it to a **job cluster**.

INTERVIEW ANGLE

Also mention **cluster policies** (restrict sizes/types for governance), **tags** for cost attribution, and **serverless compute** where available (no cluster management, fast startup).

7.5 Cluster Pools

★☆☆

IN ONE LINE

Pre-warmed idle instances that cut cluster start and scale-up latency.

A **cluster pool** keeps a set of **idle, ready-to-use instances**. When a cluster or job needs nodes it takes them from the pool instead of asking the cloud provider for fresh VMs, cutting **start-up and scale-up latency** from minutes to much less.

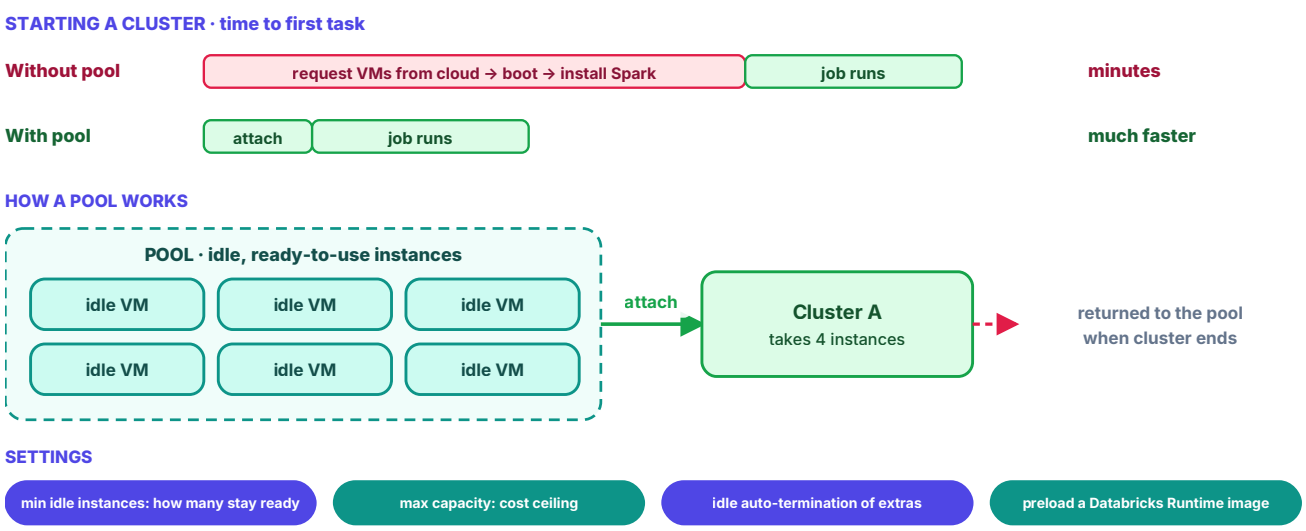


Figure 7.5 — Pools remove VM provisioning time from cluster start

Benefit	Cost / caveat
Faster job and cluster starts (many short jobs)	Idle pool instances still incur **cloud VM cost** (but no Databricks DBU charge while idle).
Faster autoscaling up	Need to size `min idle` carefully: too high wastes money.
Shared across many jobs / teams	A pool is tied to one instance type and region.

INTERVIEW ANGLE

Use pools when you run **many short jobs back-to-back**; for a single long nightly job the startup is a small share of total time and a pool is not worth it. **Serverless compute** solves the same latency problem without you managing a pool.

7.6 Databricks Runtime and Workload Configuration

★★☆

IN ONE LINE

Runtime version, flavour and cluster settings together shape performance and cost.

The **Databricks Runtime (DBR)** is the image your cluster runs: a tuned build of Apache Spark plus Delta Lake, optimised I/O, system libraries and optional Photon / ML packages. Runtime version and cluster configuration quietly decide a lot of performance and stability.

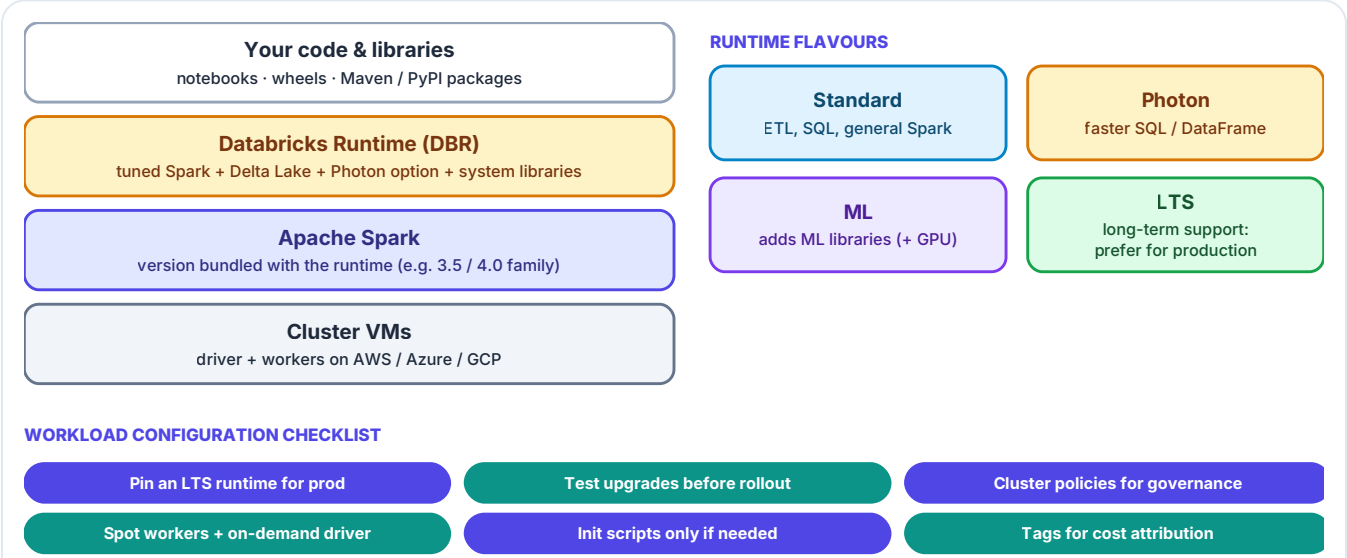


Figure 7.6 — What a runtime contains, flavours, and a configuration checklist

Decision	Guidance
Runtime version	Newer runtimes bring Spark / Delta / Photon improvements, but production should pin a tested **LTS** version and upgrade deliberately.
Runtime flavour	Standard for ETL; **Photon** for SQL-heavy workloads; **ML** for training.
Workload shape	Shuffle-heavy → memory / storage optimised nodes; CPU-bound → compute optimised; light → fewer, smaller workers.
Cost levers	Spot workers for fault-tolerant batch, job compute, auto-termination, cluster policies, tags.
Spark configs	Keep defaults unless a measurement says otherwise; document every override.

INTERVIEW ANGLE

Good phrase: *"I match compute to the workload and measure: LTS runtime, Photon if the job is SQL-heavy, right node family, spot workers for batch, and job compute with tags for cost tracking."*

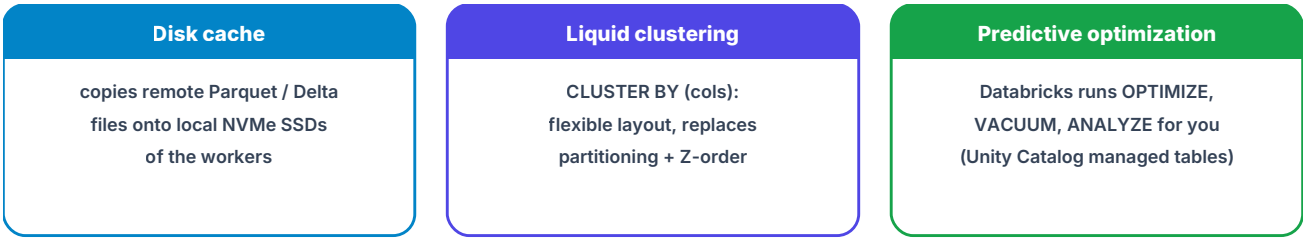
7.7 Disk Cache, Liquid Clustering and Predictive Optimization

★★☆

IN ONE LINE

Databricks storage features that automate caching, data layout and table maintenance.

Beyond compute, Databricks adds storage-layer features that remove manual tuning. Three come up often: **disk cache**, **liquid clustering** and **predictive optimization**.



	Partitioning	Z-order	Liquid clustering
Layout	folders per value	sort files on OPTIMIZE	clustered files, incremental
Change columns later	rewrite table	cheap (just re-run)	cheap: ALTER ... CLUSTER BY
High-cardinality keys	bad (tiny files)	good	good
Write skew / concurrency	ok	ok	better handled
Typical status	legacy / huge tables	widely deployed	recommended for new tables

Liquid clustering is used INSTEAD of partitioning / Z-order on the same table, not together with them.

Figure 7.7 — Three Databricks storage-layer optimisations

Feature	What it does	Notes
Disk cache (formerly Delta cache)	Automatically caches remote data files on workers' local SSD; later reads are local.	Best on storage-optimised / SSD nodes; separate from Spark `cache()`, which caches DataFrames in memory.
Liquid clustering	`CREATE TABLE … CLUSTER BY (col)` : layout is incremental and keys can change without rewriting everything.	Replaces partitioning and ZORDER for tables that use it; run OPTIMIZE (or let predictive optimization do it).
Predictive optimization	Runs maintenance (OPTIMIZE, VACUUM, statistics) automatically on **Unity Catalog managed tables**.	Reduces small-files and storage-bloat toil.

LIQUID CLUSTERING

```
-- new table with liquid clustering
CREATE TABLE sales.orders (order_id BIGINT, customer_id BIGINT, order_date DATE)
CLUSTER BY (customer_id, order_date);

-- change the clustering keys later
ALTER TABLE sales.orders CLUSTER BY (order_date);

OPTIMIZE sales.orders;      -- applies clustering incrementally
```

INTERVIEW ANGLE

If asked “how do you lay out a new Delta table on Databricks?”: *“For new tables I prefer **liquid clustering** on the common filter columns; for existing tables I may still have partitioning + Z-order; and I enable predictive optimization so compaction and vacuum happen automatically.”* Feature availability depends on runtime and edition: check the docs.